Full Stack Engineering

Project Report Semester-3 (Batch-2025)

TableBite — Restaurant Table Ordering System

Phase 2 Summary: Rebuilt in React, with a Re-render Audit



Supervised By:
Mr. Shubham Kumar
(Technical Trainer)

Submitted By:
Akshit Kumar - 2510990799 (G-16)
Parv Gupta - 2510991070 (G-16)
Armaan Dalal - 2510990717 (G-16)
Rakshit Mittal (G-16)

Department of Computer Science and Engineering
Chitkara University Institute of Engineering & Technology,
Chitkara University, Punjab

OVERVIEW

Project at a Glance

01

What it is

A restaurant ordering app: pick a table, browse a 12 or 200 item menu, add notes, see a live bill with GST and service charge, split it between people, and send tickets to a kitchen display grouped by course.

202 → 2

components re-rendered on one click at 200 menu items: naive version versus fixed version, with no memo.

Problems it solves

- One quantity change re-rendered the whole menu
- Nobody could see what re-rendered, or why
- memo everywhere hides the real cause
- No notes, split bill or table progress

What it teaches

- The parent-child re-render rule
- State placement and composition
- Measuring with the Profiler
- Keys in lists, and when memo pays

Problem Statement and Objectives

02

Problem statement

- One click re-renders the whole menu
- Speed is judged by feel, not measured
- Guests need notes, split bill, table progress

200

menu items make the re-render cost visible. With 12 items it cannot be seen.

Objectives

- Build the ordering flow in React 19
- Find what re-renders, and why
- Fix it with state placement, not blanket memo
- Prove the fix with counts and the Profiler

Scope

- In: browser app, generated menu, bill, kitchen, audit
- Out: backend, database, login, payments
- No external API is needed by the brief

Requirements and Technology Choices

03

Functional

- Menu with categories, search, diet filters
- Order with quantity and per-item notes
- Bill with GST 5%, service 5%, split
- Table states and kitchen grouped by course

React 19

with Vite, Context + useReducer, Vitest and React DevTools. Plain CSS reused from Phase 1.

Non-functional

- Fast: a click re-renders only what changed
- Correct: split shares add up to the paisa
- Tested: 13 automated tests
- Readable: small components, one job each

Why this stack

- Vite: quick start and fast reloads
- Context + useReducer: no extra library
- Vitest: can assert exact render counts
- Profiler: shows which components rendered

How It Was Built

04

1. **Study the brief**
   List every feature.
2. **Rebuild in React**
   One component per section.
3. **Add Phase 2 features**
   Notes, split bill, states.
4. **Build the audit**
   Render log and three labs.
5. **Test and measure**
   13 tests and Profiler.

Tools

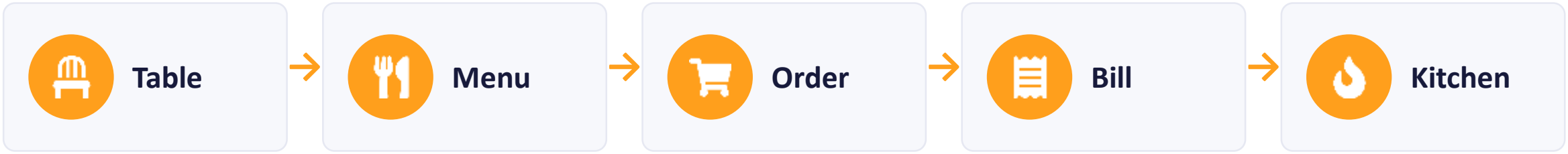
- React 19
- Vite
- Plain CSS
- Context + useReducer
- Vitest + Testing Library
- React DevTools

Same Phase 1 style.css, so the look did not change.

```
npm install
npm run dev
# http://localhost:5173
npm test
# runs the 13 tests
npm run build
# production build
```

The Ordering Flow and What Is New

05



Area	Phase 1 (HTML / CSS / JS)	Phase 2 (React)
Menu	12 items, search, filters	Plus a 200-item menu and an item details popup
Order	Quantity buttons, remove	A note on every item (for example, less spicy)
Bill	Subtotal, GST 5%, service 5%	Plus split between people, no paisa lost
Tables	Available, Occupied, Ordering	Plus Served and Billed, with buttons to move on
Kitchen	One fixed example ticket	Real orders, grouped by course, with notes
Audit and tests	Two counter buttons, manual tests	Naive, memo, fixed labs, report, 13 automated tests

State Design and Table States

06



Free Table sends a Billed table back to Available.

Context	Changes when	Read by
Actions	Never (created once)	FoodCard, Tables, Bill, Kitchen
Cart	Item, quantity or note changes	OrderList, Bill
Tables	A table changes status	Tables
Kitchen	An order is placed or advances	Kitchen

Key rule

The Menu reads none of the four contexts, so adding to the order never re-renders it.

Four contexts, one useReducer.

The Problem: The Naive Version

07

```
function NaiveLab({ items }) {
  const [qty, setQty] = useState({});

  function add(item) {
    setQty({ ...qty,
      [item.id]: (qty[item.id] || 0) + 1 });
  }

  return (
    <div>
      <NaiveSummary items={items} qty={qty} />
      {items.map(item => (
        <NaiveCard key={item.id} item={item}
          qty={qty[item.id] || 0} onAdd={add} />
      ))}
    </div>
  );
}
```

202

components re-rendered on one click (200 items): 200 cards + parent + summary

How React decides

- Its own state changes
- Its parent re-renders, so every child does too
- A context it reads changes

Two Fixes: Memo versus Composition

08

Memo version

```
const MemoCard = memo(Card);

function MemoLab({ items }) {
  const [qty, setQty] = useState({});
  const add = useCallback((item) => {
    setQty(old => ({ ...old,
      [item.id]: (old[item.id] || 0) + 1 }));
  }, []);
  return items.map(item => (
    <MemoCard key={item.id} item={item}
      qty={qty[item.id] || 0} onAdd={add} />
  ));
}
```

Fixed version (no memo)

```
function FixedLab({ items }) {
  return (
    <FixedProvider>
      <FixedSummary items={items} />
      {items.map(item =>
        <FixedCard key={item.id} item={item} />)}
    </FixedProvider>
  );
}
// state lives in FixedProvider (useReducer)
// cards read only add() from context: never changes
// only FixedSummary reads the quantities
```

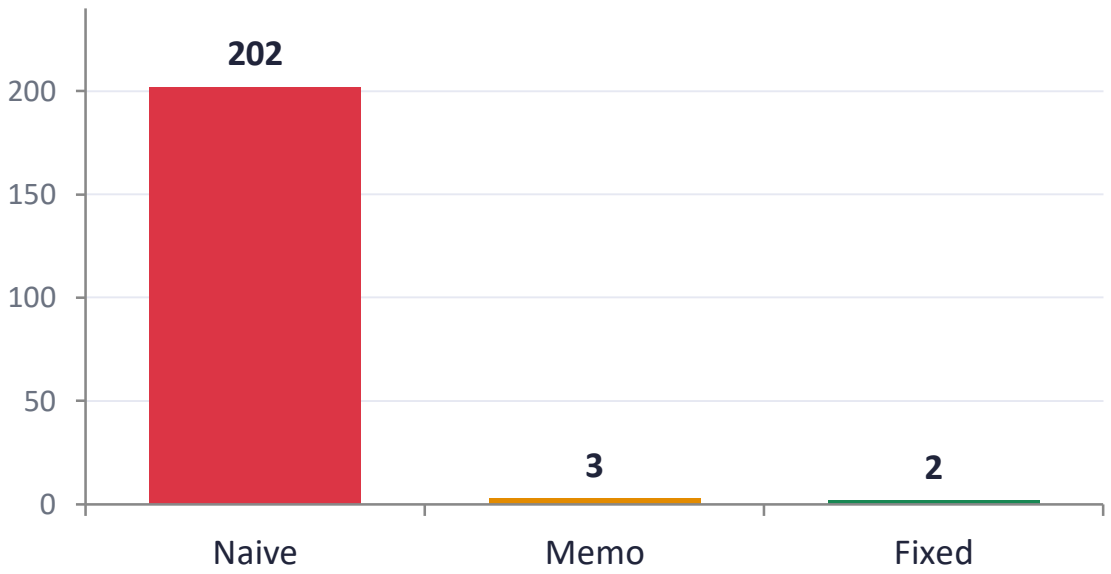
3 renders. Needs memo, useCallback and setQty(old => ...). Parent and summary still re-render.

2 renders: only the provider and the summary. The fix is where the state lives, not more hooks.

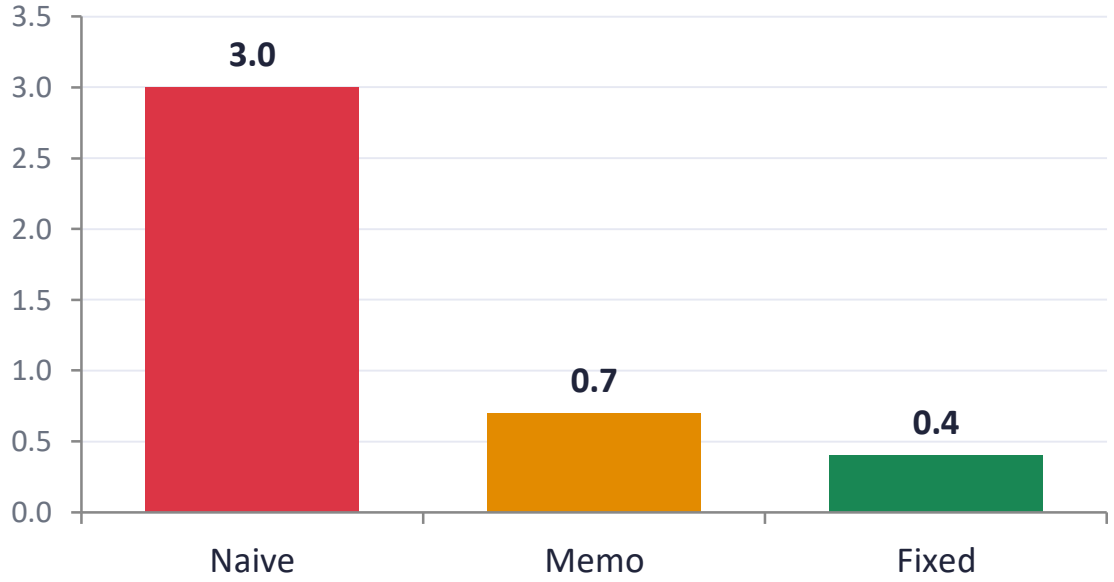
09

Measured Results at 200 Items

Components re-rendered per click (200 items)



Average time per click in ms (200 items)



Naive

Simple, but every card renders on every click.

Memo

Fast cards, but three tools and a busy parent.

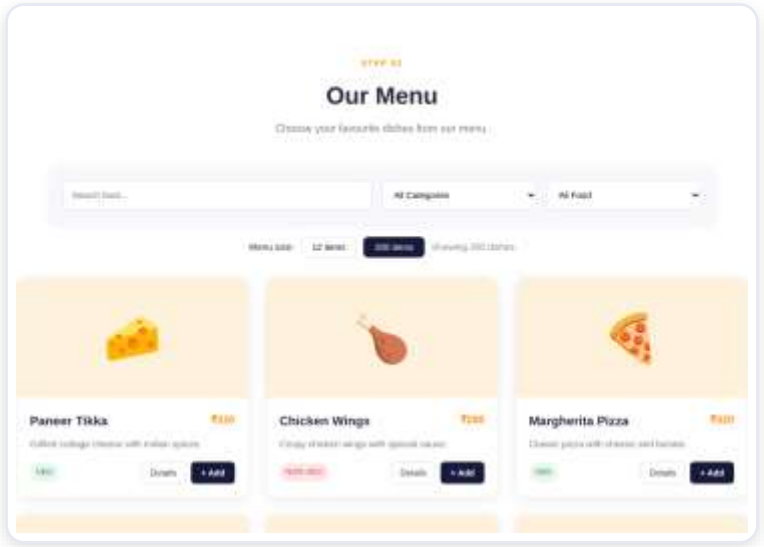
Fixed

Fastest, no memo. State placement does the work.

Counts come from the automated tests and the app render log. Times are from a test browser; my own React DevTools Profiler capture is to be added.

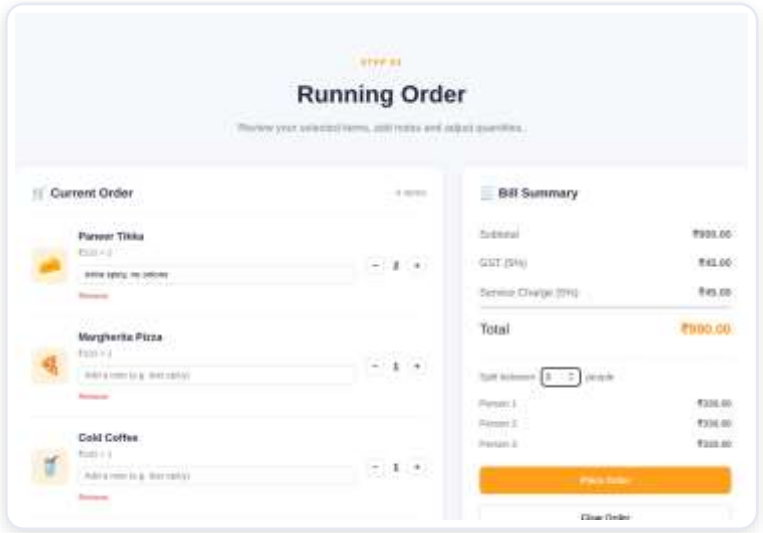
Results: The Ordering Flow

10



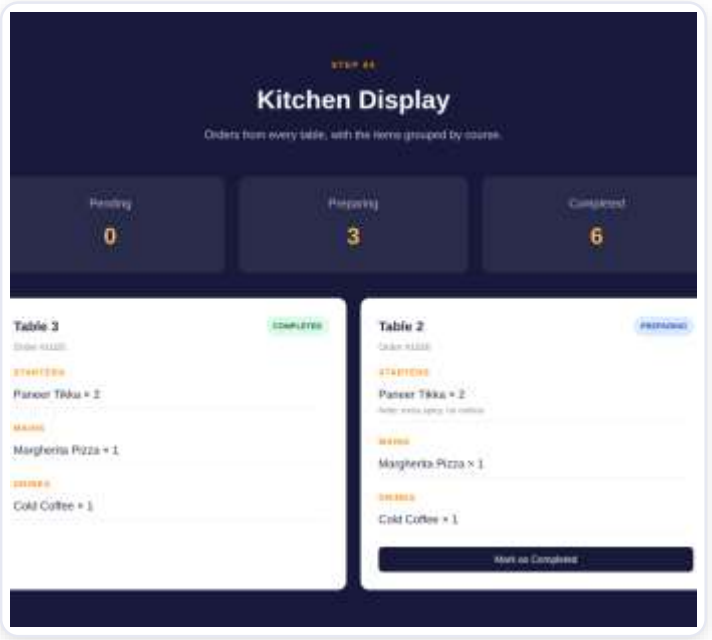
Menu, 200 items

Search, category and diet filters, size switch.



Order, bill and split

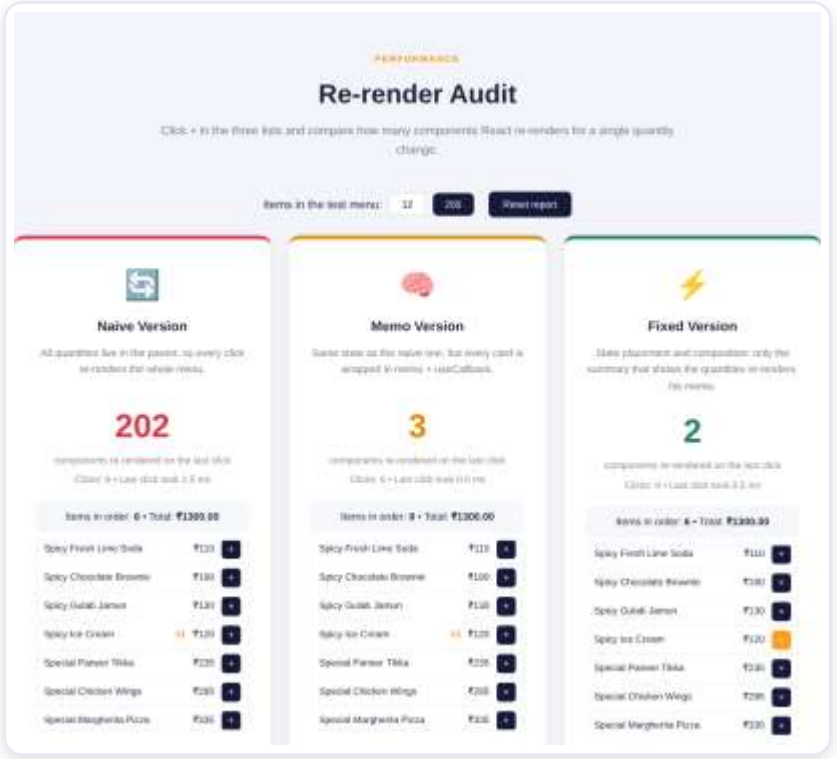
Notes, GST 5%, service 5%, split between 3.



Kitchen by course

Tickets move Pending, Preparing, Completed.

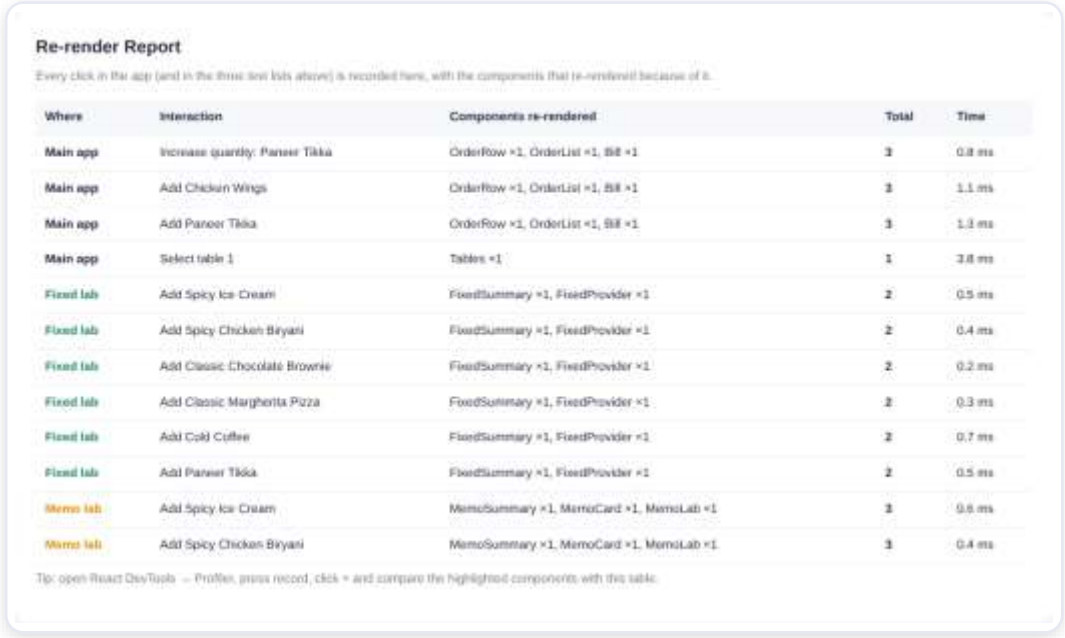
Results: Audit, Report and Keys



202
Naive

3
Memo

2
Fixed



Re-render report: which components re-rendered on each click



Keys: index key kept the note on the wrong row

key = id

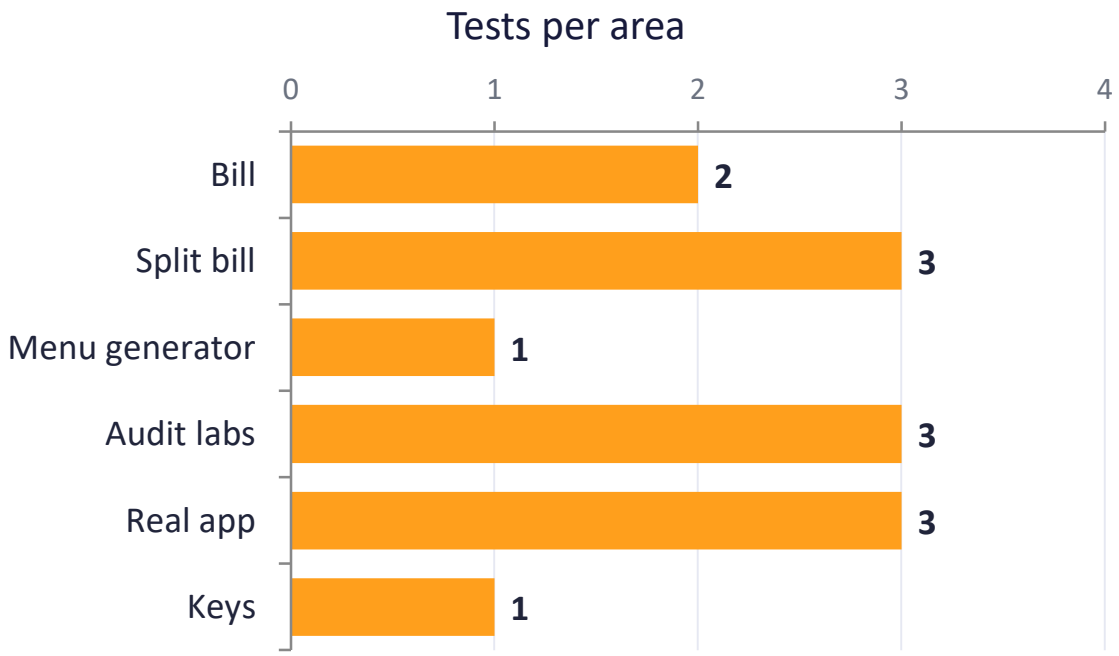
stable, correct

key = index

state sticks to the position

Testing and What I Learned

12



13 / 13 tests passing with npm test

Measure, do not guess

The render log and Profiler showed what re-rendered.

Put state where it is used

Pushing state down and splitting contexts removed re-renders.

memo has a cost

It needs stable props and extra hooks, so use it with a reason.

Keys decide row state

An index key ties state to a position, not to a dish.

Project 21 Coverage and Limits

What is done

- 200-item menu, filters, item details
- Notes, GST, service charge, split bill
- Table states and course-wise kitchen
- Naive, memo, fixed audit and 13 tests

4 open

items from the Project 21 brief are still open. They are listed under Still pending, not claimed as done.

Still pending

- Tailwind CSS: this build uses plain CSS
- Live Vercel or Netlify link
- DevTools Profiler screenshots to add
- Stretch goals: floor plan, two-window kitchen

Limitations

- Data lives in the browser: refresh resets it
- Kitchen cannot sync across devices
- Times came from a test browser, not the Profiler
- Single user: no login and no backend

Conclusion, Phase 3 and Thank You

14

Rebuilt in React

Eight sections, small components, four contexts.

Complete ordering flow

Details, notes, split bill, table states, courses.

Measured and fixed

202 re-renders became 2, with no memo.

Phase 3 roadmap

1. **Backend:** Node + Express and a database
2. **Login:** Customer and kitchen staff roles
3. **Live kitchen:** Orders sync across devices
4. **Deploy:** Vercel or Netlify with a live link

Thank You

Questions & Feedback Welcome

TableBite — Restaurant Table Ordering System
Chitkara University Institute of Engineering & Technology